_Anzam Shahriar Kabir
_2022521460123
_Neural Modeling final report

Analysis of Spike Waveforms Using PCA and K-Means Clustering

Introduction

In the realm of neuroscience, understanding the electrical activity of neurons is paramount. Neurons communicate through electrical impulses known as **spikes** or **action potentials**, which can be recorded and analyzed to gain insights into neuronal behavior. By examining these spike waveforms, researchers can identify different types of neurons, understand their interactions, and uncover patterns in neural activity. This report outlines a comprehensive approach to analyzing spike waveforms using Python, focusing on data visualization, dimensionality reduction with Principal Component Analysis (PCA), and clustering with K-Means. The analysis was conducted using Jupyter Notebook, providing an interactive environment that facilitated data exploration and visualization.

Setting Up the Environment with Jupyter Notebook

Preparing the Workspace

To perform the spike waveform analysis, the first step was to set up the working environment. The analysis was conducted using **Jupyter Notebook**, a powerful tool that allows for interactive coding, visualization, and documentation within a single platform. The process began by navigating to the directory containing the teacher-provided files using the Command Prompt (CMD).

1. **Opening Command Prompt:**
    - Press `Win + R`, type `cmd`, and hit `Enter` to open the Command Prompt.
2. **Navigating to the Project Directory:**

Use the `cd` (change directory) command to navigate to the folder containing the teacher's given files. The name in my directory was:

D:\Sichuan University\Semester 5\Neural\Anzam_2022521460123 final project\Final Report of NMDA-PCA\Final Report of NMDA-PCA

    - Replace the path with the actual directory where the `spikewave.mat` file and related scripts are located.
3. **Launching Jupyter Notebook:**

Once inside the desired directory, launch Jupyter Notebook by typing:
jupyter notebook

    - This command opens Jupyter Notebook in the default web browser, displaying the contents of the current directory.

4. **Creating a New Notebook:**

    - In the Jupyter interface, click on `New` and select `Python 3` to create a new notebook for the analysis.

Advantages of Using Jupyter Notebook

Jupyter Notebook offers several benefits for data analysis:

- **Interactive Coding:** Allows for running code in small chunks, making it easier to debug and understand.
- **Visualization:** Supports inline plotting, enabling immediate visualization of data and results.
- **Documentation:** Facilitates combining code with markdown text, making the analysis well-documented and easy to follow.
- **Reproducibility:** Ensures that the analysis steps are preserved and can be replicated or shared with others.

Data Loading and Preparation

Loading Spike Data

The spike waveform data was provided in a MATLAB `.mat` file named `spikewave.mat`. MATLAB files are commonly used in scientific research to store complex data structures. To load and manipulate this data in Python, the `scipy.io` library was utilized.

```
import scipy.io as sio
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import numpy as np

# Load the spike data from the .mat file
data = sio.loadmat('spikewave.mat')

# Check the structure of the data
print(data)
```

Inspecting the Data Structure

We may examine the contents of the.mat file by printing the data dictionary. Typically, the spike waveforms are stored under a specific key, such as 'wave'. To properly extract the data, one must comprehend the structure.

```
# Extract spike waveforms
spike_data = data['wave'].T
```

Transposing the array ensures that each row represents a single spike, and each column corresponds to the amplitude at a specific time point.In this project, the correct format to use is 5376 spike waveforms, each with a length of 40 sampling points. The original .mat data needs to be transposed. So it will be (40, 5376).

Understanding the Data

Each spike waveform captures the neuron's electrical activity over time. By analyzing these waveforms, we can identify patterns or variations that may indicate different types of neurons or distinct firing behaviors.

Visualizing Spike Waveforms

Before delving into complex analyses, visualizing the raw spike waveforms provides an initial understanding of the data's variability and patterns.

```
# Plot the first 10 spike waveforms
plt.figure(figsize=(10, 6))
for i in range(10):
    plt.plot(spike_data[i, :], label=f'Spike {i+1}')
plt.title("Spike Waveforms")
plt.xlabel("Time (samples)")
plt.ylabel("Amplitude")
plt.legend(loc='upper right')
plt.show()
```

Interpretation of the Plot

The plot displays the amplitude of the first 10 spikes over time. Observing these waveforms helps identify patterns or variations among different spikes. For example:

- **Amplitude Differences:** Some spikes may have higher or lower amplitudes, indicating varying neuronal firing strengths.
- **Duration Variations:** The length of the spike waveform can suggest differences in how quickly neurons fire.
- **Shape Distinctions:** Unique shapes or features in the waveforms may signify different neuron types or functional states.

Dimensionality Reduction with PCA

Why should we reduce Dimensions?

Spike waveforms are high-dimensional data, particularly when multiple time points are captured for each spike. Visualising and analysing high-dimensional data can be difficult. While maintaining as much variance as possible, Principal Component Analysis (PCA) assists in reducing the dimensionality of the data. Clustering and data visualisation are facilitated by this simplification.

Performing PCA

PCA transforms the original data into a set of new variables called **principal components**. These components capture the most significant variations in the data.

```
# Standardize the data
scaler = StandardScaler()
spike_data_scaled = scaler.fit_transform(spike_data)

# Perform PCA to reduce the dimensionality to 2D and 3D
pca_2d = PCA(n_components=2)
pca_3d = PCA(n_components=3)

# Transform the spike data
spike_data_2d = pca_2d.fit_transform(spike_data_scaled)
spike_data_3d = pca_3d.fit_transform(spike_data_scaled)

# Explained variance
print(f"Explained variance by PCA components (2D): {pca_2d.explained_variance_ratio_}")
print(f"Explained variance by PCA components (3D): {pca_3d.explained_variance_ratio_}")
```

Standardizing the Data

Before applying PCA, the data was standardized using `StandardScaler`. Standardization ensures that each feature (time sample) has a mean of 0 and a standard deviation of 1. This step is crucial because PCA is sensitive to the scale of the data.

Visualizing PCA Results

2D PCA Plot

Reducing the data to two principal components allows for straightforward visualization.

```
# 2D PCA Plot
plt.figure(figsize=(8, 6))
plt.scatter(spike_data_2d[:, 0], spike_data_2d[:, 1], alpha=0.7)
plt.title("PCA - 2D")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()
```

3D PCA Plot

Including a third principal component provides a more comprehensive view of the data's structure.

from mpl_toolkits.mplot3d import Axes3D

```
# 3D PCA Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(spike_data_3d[:, 0], spike_data_3d[:, 1], spike_data_3d[:, 2], alpha=0.7)
ax.set_title("PCA - 3D")
ax.set_xlabel("Principal Component 1")
ax.set_ylabel("Principal Component 2")
ax.set_zlabel("Principal Component 3")
plt.show()
```

Understanding PCA Plots

- **Principal Components:** The axes in PCA plots represent principal components, which are new variables that capture the maximum variance in the data. The first principal component (PC1) captures the most variance, followed by the second (PC2), and so on.

- **2D vs. 3D:** The 2D plot shows the data projected onto the first two principal components, while the 3D plot includes the third component. These plots help visualize the spread and potential clustering of spike waveforms in reduced dimensions.

Clustering with K-Means

Introduction to K-Means

K-Means Clustering is an unsupervised machine learning algorithm used to partition data into distinct groups (clusters) based on similarity. The algorithm assigns each data point to the nearest cluster centroid, iteratively refining the centroids to minimize the within-cluster variance.

Applying K-Means

We apply K-Means clustering to both the 2D and 3D PCA-transformed data to identify groups of similar spike waveforms.

```
# Apply KMeans clustering
n_clusters = 4  # Adjust as needed
kmeans_2d = KMeans(n_clusters=n_clusters, random_state=42)
clusters_2d = kmeans_2d.fit_predict(spike_data_2d)
```

```
kmeans_3d = KMeans(n_clusters=n_clusters, random_state=42)
clusters_3d = kmeans_3d.fit_predict(spike_data_3d)
```

Visualizing Clusters

2D Plot with Clustering

```
# 2D Plot with Clustering
plt.figure(figsize=(8, 6))
plt.scatter(spike_data_2d[:, 0], spike_data_2d[:, 1], c=clusters_2d, cmap='viridis', alpha=0.7)
plt.title("PCA - 2D with Clusters")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()
```

3D Plot with Clustering

```
# 3D Plot with Clustering
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(spike_data_3d[:, 0], spike_data_3d[:, 1], spike_data_3d[:, 2],
                     c=clusters_3d, cmap='viridis', alpha=0.7)
ax.set_title("PCA - 3D with Clusters")
ax.set_xlabel("Principal Component 1")
ax.set_ylabel("Principal Component 2")
ax.set_zlabel("Principal Component 3")
plt.show()
```

Interpreting Clustered Plots

- **Color Coding:** Each cluster is represented by a different color, making it easy to distinguish between groups.
- **Cluster Separation:** Well-separated clusters indicate that the K-Means algorithm effectively grouped similar spike waveforms together. Overlapping clusters might suggest that some waveforms share similar characteristics or that more clusters are needed.
- **Centroids:** Although not explicitly plotted here, the centroids represent the average position of all points within a cluster in the PCA space.

Sorting and Plotting Clusters Individually

After clustering, it's beneficial to examine the waveforms within each cluster to understand their characteristics.

```
# Sort the spikes by their cluster assignments
sorted_spikes = [spike_data[clusters_2d == i] for i in range(n_clusters)]

# Plot each cluster with a different color
for i, cluster in enumerate(sorted_spikes):
    plt.figure(figsize=(10, 6))
    for spike in cluster:
        plt.plot(spike, color=plt.cm.viridis(i / n_clusters))
    plt.title(f"Cluster {i+1}")
    plt.xlabel("Time (samples)")
    plt.ylabel("Amplitude")
    plt.show()
```

Insights from Clustered Waveforms

By plotting the waveforms within each cluster:

- **Cluster Consistency:** Waveforms within a cluster should exhibit similar shapes, amplitudes, and durations, reflecting consistent neuronal activity patterns.
- **Variability:** Some clusters might show more variability, indicating a mix of slightly different spike types or noise in the data.
- **Distinct Features:** Certain clusters might display unique features, such as spikes with higher amplitudes or longer durations, which can be linked to specific neuronal properties.

Additional Considerations

Determining the Optimal Number of Clusters

Choosing the right number of clusters (`n_clusters`) is crucial for meaningful analysis. While four clusters were used in this example, other methods can help determine the optimal number:

Elbow Method

Plotting the within-cluster sum of squares, or inertia, against the number of clusters is the Elbow Method. An ideal number is suggested by the "elbow point" where the rate of decline changes significantly.

```
# Elbow Method to find the optimal number of clusters
inertia = []
K_range = range(1, 10)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(spike_data_2d)
    inertia.append(kmeans.inertia_)
```

```
plt.figure(figsize=(8, 5))
plt.plot(K_range, inertia, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal k')
plt.xticks(K_range)
plt.grid(True)
plt.show()
```

Silhouette Score

The Silhouette Score measures how similar a point is to its own cluster compared to other clusters. Higher scores indicate better-defined clusters.

```
from sklearn.metrics import silhouette_score

silhouette_scores = []
for k in range(2, 10):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(spike_data_2d)
    score = silhouette_score(spike_data_2d, labels)
    silhouette_scores.append(score)

plt.figure(figsize=(8, 5))
plt.plot(range(2, 10), silhouette_scores, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Method for Optimal k')
plt.xticks(range(2, 10))
plt.grid(True)
plt.show()
```

Handling `.mat` File Structures

MATLAB `.mat` files can store data in complex structures, including nested arrays or cell arrays. It's essential to understand the data structure to extract the required variables correctly. Using `print(data)` helps inspect the keys and structures within the file. If data is nested, additional processing might be necessary to flatten or reshape the arrays for analysis.

Data Preprocessing

Before applying PCA and K-Means, preprocessing steps like **standardization** enhance the analysis:

```
from sklearn.preprocessing import StandardScaler
```

```
# Standardize the data
scaler = StandardScaler()
spike_data_scaled = scaler.fit_transform(spike_data)
```

Standardizing ensures that each feature (time sample) has a mean of 0 and a standard deviation of 1. This step is vital because PCA and K-Means are sensitive to the scale of the data.

Saving Plots

To document the analysis, saving plots can be useful:

plt.savefig('cluster_plot.png')

Adding this line before `plt.show()` saves the current plot as an image file. This practice is beneficial for creating reports or presentations based on the analysis.

Conclusion

This analysis demonstrates a systematic approach to processing and analyzing neuronal spike waveforms using Python within the Jupyter Notebook environment. By loading the data from a .mat file, visualizing raw waveforms, reducing dimensionality with PCA, and applying K-Means clustering, we can categorize spike waveforms into meaningful groups. These clusters reveal underlying patterns in neuronal activity, potentially distinguishing different types of neurons or identifying specific firing behaviors.

Key Takeaways:

1. **Interactive Environment:** Using Jupyter Notebook facilitates an interactive and organized workflow, allowing for seamless integration of code, visualization, and documentation.
2. **Data Visualization:** An basic comprehension of the variability and patterns in the data can be obtained by plotting raw spike waveforms.
3. **Dimensionality Reduction:** PCA effectively reduces high-dimensional data to lower dimensions, facilitating visualization and clustering while retaining essential variance.
4. **Clustering:** K-Means groups similar spike waveforms, aiding in the identification of distinct neuronal populations or activity patterns.
5. **Optimization:** Techniques like the Elbow Method and Silhouette Score assist in determining the optimal number of clusters, ensuring meaningful and interpretable results.

Future Directions:

- **Advanced Clustering Algorithms:** Exploring algorithms like DBSCAN or hierarchical clustering could provide deeper insights, especially in datasets with non-spherical cluster

shapes.

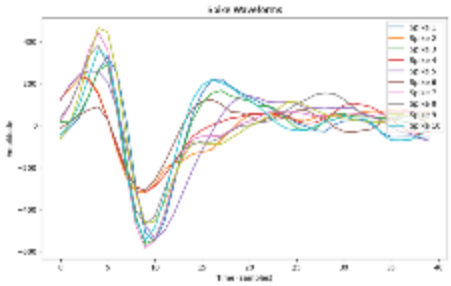
- **Feature Engineering:** Incorporating additional features, such as spike duration or amplitude, could enhance clustering performance.

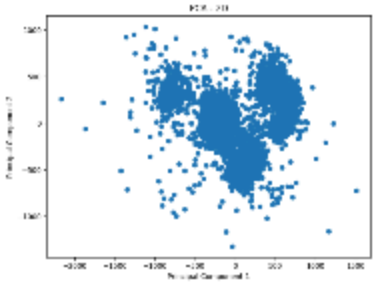
- **Validation:** Using labeled data or expert annotations can help validate the clustering results, ensuring that the groups identified are biologically meaningful.

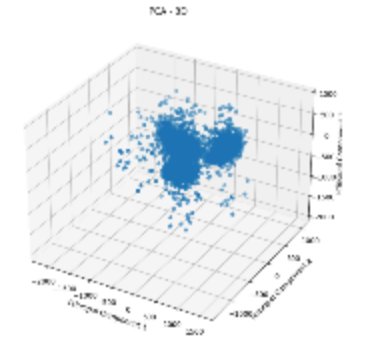
- **Real-Time Analysis:** Implementing the analysis pipeline in real-time could aid in live neuronal monitoring and applications like brain-machine interfaces.

In summary, the combination of PCA and K-Means within the Jupyter Notebook environment provides a powerful toolkit for neuronal spike analysis. This approach not only aids in understanding neuronal behavior but also contributes to advancements in neuroscience research and related applications.

The final summary of the output :


Spike Waveforms




PCA - 3D